ABOUT US

Trainocate is one of the leading IT Training providers across the globe. With over 30 years in the industry, presence in 24 different countries, and various global awards, it has proven to be an expert in its field, trusted by many clients across various industries.

CONTACT US

ph.inquries@trainocate.com

www.trainocate.com/ph

CONNECT WITH US

www.facebook.com/TrainocatePH

https://www.linkedin.com/company/trainocateph/

VISIT US

Unit 1202, MDI Corporate Center, 10th ave., cor 39th st. Bonifacio Global City, Taguig, Metro Manila, Philippines 1634

TRAINOCATE

Fundamentals of AI Driven Software Development with Claude

Duration: 1 Day

OBJECTIVES

By the end of the day, participants will understand the core building blocks of a web application and use Claude to plan, modify, run, test, and improve a backend API and frontend admin panel.

Learning Outcomes:

- Explain what an API, server, database, and Docker container are in practical terms.
- Understand how a frontend communicates with a backend API.
- Run and inspect a template-based full-stack web application locally.
- Use Claude to explore an unfamiliar codebase, plan changes, implement features, debug errors, and verify results.
- Create project guidance using an instruction file such as AGENTS.md or CLAUDE.md so Claude consistently follows project conventions.
- Use Claude skills for repeatable development workflows such as feature implementation, code review, debugging, and verification.

DEFAULT PROJECT TEMPLATES

- **Backend:** cloudband-solutions/default_api_fast
- **Frontend:** cloudband-solutions/admin-panel-web

TOOLS SETUP

- **VS Code setup:** Install VS Code, configure the integrated terminal, add recommended extensions for Python, frontend development, Docker, Git, and code formatting, and confirm participants can open both repositories as workspaces.
- **Python setup:** Install the required Python version, create and activate a virtual environment, install backend dependencies, and confirm the backend can run locally.
- **Docker setup of PostgreSQL:** Install Docker Desktop, confirm Docker is running, start a PostgreSQL container or Docker Compose service, verify the database port and credentials, and confirm the backend can connect to the database.
- **Running the default repositories:** Clone the backend and frontend repositories, install dependencies, configure environment variables, start both apps, and confirm the frontend can call the backend API.
- **Setting up Claude:** Install and sign in to the Claude development environment, open the repositories, confirm Claude can read the project files, and prepare project instructions for consistent development behavior.

CONTENT

Module 1: Core Concepts and Local Application Foundations

Subtopic: Orientation and Mental Model

- Introduce the full-stack application participants will build on top of.
- Explain the roles of the browser, frontend app, backend API, database, and container runtime.
- Walk through the request-response cycle: user clicks a button, frontend sends a request, backend processes it, database stores or retrieves data, frontend updates the screen.
- Activity: Draw the application architecture on a whiteboard or shared diagram.

Subtopic: What Is an API?

- Define an API as a contract that allows software systems to communicate.
- Explain endpoints, HTTP methods, request bodies, query parameters, headers, and response codes.
- Use simple examples such as GET users, POST create item, PUT update item, and DELETE item.
- Discuss JSON as the common data format between frontend and backend.
- Hands-on: Inspect sample API routes in the backend template and call one endpoint using a browser, API client, or terminal command.

Subtopic: Server, Backend, and Business Logic

- Explain what a server does: receives requests, validates input, runs logic, talks to services or databases, and returns responses.
- Introduce route handlers, controllers, services, schemas, and configuration.
- Show how backend code is organized in the default API template.
- Discuss environment variables and why secrets should not be hard-coded.
- Hands-on: Trace one backend request from route definition to response.

Subtopic: Database Basics

- Define a database as the persistent storage layer of the application.
- Explain tables, records, fields, primary keys, relationships, and basic CRUD operations.
- Connect API endpoints to database operations.
- Discuss migrations, seed data, and why database structure matters for application behavior.
- Hands-on: Identify where database configuration, models, or data access code lives in the backend template.

Subtopic: Simple Docker Concepts

- Explain containers as isolated, repeatable environments for running software.
- Clarify the difference between an image, a container, a Dockerfile, and Docker Compose.
- Show how Docker helps avoid “works on my machine” problems.
- Discuss common commands: build, up, down, logs, exec, and restart.
- Hands-on: Start the backend, frontend, and database services locally using the provided setup.

Module 2: Building with Claude

Subtopic: Working with Claude as a Development Partner

- Position Claude as a repo-aware development assistant, not just a code generator.
- Teach the basic loop: ask Claude to explore, ask for a plan, approve or revise the plan, let Claude implement, then verify.
- Emphasize giving Claude clear success criteria such as tests, build commands, expected UI behavior, and sample API responses.
- Discuss safety: avoid pasting secrets, review generated code, keep changes small, and commit working checkpoints.

Subtopic: Codebase Exploration with Claude

- Prompt Claude to summarize the backend template structure: routes, configuration, database layer, dependencies, and run commands.
- Prompt Claude to summarize the frontend template structure: pages, components, API client, state management, routing, and styling.
- Ask Claude to map how the frontend calls the backend.
- Activity: Participants create a short "project map" in their notes that explains where to add a new feature.

Subtopic: Project Instructions with AGENTS.md or CLAUDE.md

- Explain that a project instruction file gives Claude persistent guidance about the repository, commands, coding standards, and constraints.
- Discuss what belongs in the file: project overview, architecture, local run commands, test commands, style rules, database rules, API conventions, frontend conventions, and "do not" instructions.
- Keep instructions concise and specific so Claude has useful context without unnecessary noise.
- Activity: Participants draft an instruction file that tells Claude how to work with the backend and frontend templates.

Sample Instruction File Sections

- **Project purpose:** This repository contains a backend API and frontend admin panel for a full-stack web application.
- **Backend commands:** Include commands for installing dependencies, running the API, running tests, applying migrations, and starting services with Docker.
- **Frontend commands:** Include commands for installing dependencies, starting the dev server, building the app, linting, and running tests.
- **Coding conventions:** Follow existing folder structure, naming patterns, validation approach, and error handling style.
- **Verification:** After any change, run the smallest relevant check first, then run broader tests or builds when appropriate.
- **Change discipline:** Make focused changes, explain assumptions, avoid unrelated refactors, and ask before introducing new major dependencies.

Subtopic: Using Claude Skills for Repeatable Workflows

- Explain skills as reusable instructions that Claude can load when a task matches a workflow.
- Discuss when to create a skill: repeated feature checklist, release process, debugging routine, code review standard, API endpoint creation, or frontend component creation.

- Show the typical skill structure: a skill folder, a SKILL.md file, concise instructions, and optional scripts or reference files.
- Activity: Draft a simple “Add API Endpoint” skill that tells Claude to inspect existing patterns, add schema validation, implement route logic, update tests, and verify the endpoint.

Subtopic: Guided Build Exercise with Claude

- Choose a small feature that touches both backend and frontend, such as adding a simple entity list, creating a new admin page, or adding a filter field.
- Prompt Claude to inspect the current implementation and propose a plan before editing.
- Have Claude implement the backend API changes first.
- Have Claude implement the frontend UI and API integration.
- Run verification steps: backend tests or API checks, frontend build or lint, and manual browser walkthrough.
- Ask Claude to summarize what changed and identify remaining risks or follow-up improvements.

Subtopic: Best Practices for Claude-Assisted Development

- **Start with context:** Tell Claude the goal, relevant files, constraints, and expected outcome.
- **Ask for a plan first:** Require Claude to explain intended changes before implementation.
- **Keep tasks small:** Build one endpoint, one component, or one workflow at a time.
- **Give verification criteria:** Provide tests, commands, expected response shapes, and acceptance criteria.
- **Use existing patterns:** Ask Claude to inspect and follow the repository’s current conventions.
- **Control dependencies:** Ask Claude to justify any new package before adding it.
- **Review generated code:** Check validation, error handling, security, accessibility, and maintainability.
- **Commit frequently:** Save stable milestones before asking Claude to make larger changes.
- **Document decisions:** Ask Claude to update notes or instruction files when the project gains new conventions.

AUDIENCE

- New or early-career developers, technical analysts, product builders, and team members who want to use Claude as a practical software development partner while building a full-stack web application from default templates.